Week 4, Day 7 (Weekend): System Design of a Rate Limiter, Week 4 Mock and Review

Prepared for Upendra Kumar Sharma | 3.5 to 4 hours | Main topics: designing a distributed rate limiter, the Week 4 mock interview, project checkpoint, Week 5 preparation

Today's Schedule

- **0:00 to 0:20** Re-read the seven-step HLD framework from Week 3, Day 7, and the rate limiting algorithms from Week 4, Day 4.
- **0:20 to 1:05** Read the rate limiter walkthrough (section 1).
- **1:05 to 1:50 Practice:** close this guide and design the rate limiter aloud in 35 minutes. Then compare and note what you missed (section 2).
- **1:50 to 2:40** Take the Week 4 mock interview (section 3).
- **2:40 to 3:10** Project checkpoint and your Week 4 review (sections 4 and 5).
- **3:10 to 3:30** Preparation for Week 5 (section 6).

1. Worked Design: A Distributed Rate Limiter

The prompt: design a rate limiter that protects an API used by millions of clients. It must work across many servers and decide quickly whether each request is allowed.

Step 1: Requirements

- **Functional**
  - Limit requests per **client** (user id, API key or IP address), and optionally per endpoint, with different limits for different plans.
  - Reject excess requests with **HTTP 429**, and tell the client when to retry.
  - Rules are **configurable** and changeable without a deployment.
  - Allow short bursts if the rule says so.
- **Non-functional**
  - **Low latency overhead:** the check must add only a few milliseconds, because it sits on every request.
  - **Highly available:** the limiter must never become the reason the API is down. If it fails, **let traffic through** (fail open).
  - **Scalable:** tens of thousands of checks per second and millions of distinct keys.

    - **Accurate enough:** soft limits may be slightly off, hard limits (billing, security) need more care. Ask which one this is.
    - **Distributed:** the same limit must hold across all API servers.
- **Out of scope:** network-level DDoS protection (that is a job for a CDN or WAF).

Step 2: Estimation (assumptions)

- 10 million daily active users, and about 100 requests each per day, is **1 billion requests per day**, about **12,000 per second** on average and roughly **40,000 per second** at peak.
- **State per key:** a token bucket needs two numbers (tokens and last refill time), about 150 bytes with key and overhead. For 10 million active keys that is about **1.5 GB**, which fits in memory.
- **Throughput:** one Redis node handles on the order of 100,000 simple operations per second, and Lua scripts cost more. So **40,000 checks per second fits a single node technically, but you want several shards for headroom and availability.**

Step 3: Where to Place the Limiter

- **In the client:** cannot be trusted, so it is only a courtesy.
- **Inside each service as middleware or a library:** simple, but every service must implement it, and per-instance counters need a shared store anyway.
- **At the API gateway (the usual choice):** one place, applied before traffic reaches services, and it can be enforced for every route.
- **As a separate rate limiter service:** flexible, but adds a network hop to every request.
- **A good answer:** enforce it in the **gateway or middleware**, backed by a **shared Redis store**, with rules loaded from a configuration service.

Step 4: Choose the Algorithm

- **Token bucket** (the default choice): allows controlled bursts, needs little memory (two values per key), and is easy to make atomic. Used by many large API providers.
- **Sliding window counter:** more accurate over a rolling window with only two counters per key, with a small approximation.
- **Sliding window log:** exact, but stores a timestamp per request, so memory grows with the limit.
- **Fixed window:** simplest, but has the boundary burst problem (up to twice the limit across a window edge).
- **Say:** "I would use a token bucket for most limits, and a sliding window counter where the rolling-window accuracy matters."

Step 5: High-Level Architecture

```
Client --> Load balancer --> Gateway nodes (limiter middleware + rules cache)
                                  |                     |
                                  |                     +--> Rules / config
service (loaded and cached)
                                  v
                             Redis shards (token bucket state)
                                  |
                                  v (allowed requests only)
                             Upstream services
```

Request flow

1. The gateway identifies the client (API key, user id from the token, or IP) and the endpoint.
2. It looks up the matching **rule** in its **local cache** (limit, refill rate, burst, key type).
3. It runs one **atomic Lua script** on the Redis shard that owns the key. The script refills the bucket from the elapsed time, takes a token if available, and returns allowed or denied together with the remaining tokens.
4. If **allowed**, it forwards the request and adds the **X-RateLimit-Limit** and **X-RateLimit-Remaining** headers. If **denied**, it returns **429** with **Retry-After** (and the reset time).
5. It records metrics (allowed and denied counts by rule).

Data and rules

- **Key:** a namespaced string such as rl:{rule}:{clientId}, with a TTL slightly longer than the time to refill a full bucket, so idle keys disappear.
- **Rules** (stored in a database or a Git-backed config) look like: domain, key type, limit per period, burst capacity, and plan. Gateways **cache them in memory** and refresh them periodically or on change.

Step 6: Deep Dive

Atomicity and races

- Reading the counter, deciding, and writing it back in separate commands lets two concurrent requests both pass. Use a **single Lua script** (or an atomic command such as INCR), so the whole check-and-update is one step on the shard.
- Keep each limiter to **one key**, so the script runs on a single Redis Cluster slot.

Sharding and the failure of a shard

- Spread keys across shards with **consistent hashing** or Redis Cluster hash slots. Add replicas for availability.

- Replication is **asynchronous**, so after a failover some counters may be lost or reset. For a **soft limit** that is acceptable (a client briefly gets a few extra requests). For a **hard limit** (billing, security), accept the cost of stronger guarantees or count in a durable store.

Reducing load on Redis

- **Local batching:** each gateway node takes a **slice of tokens** from Redis for a key (for example 10 at a time) and serves requests from memory until the slice is used. Fewer round trips, with slightly less accurate global limits. Say that you are trading accuracy for latency and load.
- **Local deny cache:** once a client is denied, remember that locally until the retry time, so an abusive client's flood stops at the gateway and never reaches Redis.

Hot keys: one abusive or very large client concentrates traffic on one shard. The local deny cache and local batching handle most of it. For a legitimately huge client, split its limit across several sub-keys.

Time: nodes' clocks can drift. Use the Redis **TIME** command inside the script (or store only relative values), instead of trusting each gateway's clock.

Multiple regions

- A **global** limit needs cross-region coordination, which costs latency. The usual compromise is **per-region limits** (each region gets a share of the quota), or asynchronous synchronization of counters with a little over-admission. State the trade-off.

Step 7: Failure Handling, Operations and Trade-offs

- **Redis slow or down:** use a **very short timeout** (tens of milliseconds) and a **circuit breaker**. Then **fail open**, and apply a coarse **local in-memory limiter** (a fraction of the limit per node) so some protection remains. Alert on it. For sensitive endpoints such as login or payment, you may choose to fail closed.
- **Config service down:** keep using the **last known rules**.
- **Recovery:** avoid a stampede when Redis comes back (jittered retries, gradual re-enabling).
- **Safe rollout of new rules:** run a new rule in **shadow (log-only) mode** first, compare what it would have blocked, and then enforce it.
- **Observability:** allowed and denied counts per rule, limiter latency, Redis errors, and top denied keys. Use them to spot abuse and wrong limits.
- **Other features to mention:** weighted or **cost-based limits** (expensive endpoints cost more tokens), per-plan tiers, a separate **long-term quota** (monthly) kept in durable storage, allow-lists for trusted internal callers, and combining signals (IP plus account)

against abuse.
- **Client side:** honor Retry-After, and use exponential backoff with jitter. Good API docs state the limits.

Likely follow-up questions

1. How do you stop two concurrent requests from both taking the last token?
2. How do you keep the limit exact across many gateways, and what does exactness cost?
3. What if Redis fails? Fail open or closed?
4. How do you handle one client that floods the system?
5. How would you support different limits per plan and per endpoint?
6. How would you implement a monthly quota alongside the per-second limit?
7. How would you roll out a new limit without breaking customers?
8. How would this work across regions?

2. Practice Exercise (45 Minutes)

1. Close this guide. Take a blank page and a timer.
2. Design the rate limiter aloud for **35 minutes**, following the seven steps. Record yourself if you can.
3. Compare with section 1. Write down **three things you missed** and **one thing you explained better than the guide**.
4. **Common mistakes to check yourself against:** starting to draw before asking questions, skipping the numbers, picking tools without saying why, ignoring failure of the limiter itself, and never discussing trade-offs.
5. Repeat later this week with a different prompt (a notification service, which is Week 5, Day 7, is a good candidate).

3. Week 4 Mock Interview (40 Minutes)

Answer each in 90 seconds or less. Score yourself: 2 = complete, 1 = partial, 0 = could not answer.

Kafka

1. What happens between calling send() and the record being acknowledged? Why is send() asynchronous?
2. Explain acks, min.insync.replicas and what makes a configuration durable. Why can acks=all still lose data?

3. What does the idempotent producer guarantee, and what does it not?
4. How does Kafka guarantee ordering? How can you break it?
5. Explain a rebalance: what triggers it, what it costs, and how to reduce it.
6. How do you handle failures in a consumer? Compare blocking retries, retry topics and a dead letter topic.
7. How do you get effectively-once results when a consumer writes to a database?
8. Consumer lag keeps growing. What do you do?

Redis and caching

9. Explain cache-aside. Why delete rather than update on write? Describe the stale data race.
10. What are a cache stampede, penetration and avalanche? How do you prevent each?
11. Compare fixed window, sliding window and token bucket rate limiting. Why is a Lua script needed?
12. How does a Redis distributed lock work, and why can it still fail? What is a fencing token?

Testing and project

13. How do you test an event-driven system? Why Testcontainers instead of mocks or H2?
14. How do you prove a consumer is idempotent, and that the outbox is atomic?
15. Walk me through your project: the architecture, the three hardest problems, and what you would change.
16. What happens in your system if Kafka is down when an order is placed?

Coding and design (not scored, but do them)

- Solve one without help in 20 minutes: Kth Largest Element, Find Median from Data Stream, Koko Eating Bananas, or Implement Trie.
- Do a 5-minute outline of how you would add a payment service with compensation to your project.

Scoring (questions 1 to 16, maximum 32)

- 26 to 32: strong. Fix the few weak spots.
- 20 to 25: decent. Reread the day guides for each 0 and 1.
- Below 20: spend extra time on those topics before Week 5.

4. Project Checkpoint

By now the project should be complete. Confirm each item.

- [ ] A fresh clone runs with one command, following only the README.
- [ ] The tests pass locally and in GitHub Actions (green badge).
- [ ] The README has diagrams, design decisions, failure drill results and your measured numbers.
- [ ] At least three ADRs are written, and the repository is tagged and pinned.
- [ ] You can give the 3-minute pitch and answer the ten project questions.

If something is missing, finish it now. Before applying (planned for Week 6), send me your real numbers (tests, latency, drill count), and I will update the resume with the Projects section.

5. Week 4 Review (Write in Your Notes)

- What was my mock score, and which topics scored 0 or 1?
- Which behavior did I only understand after running it (a rebalance, a stampede, a breaker opening, a duplicate message)?
- Which DSA problems this week do I still not trust myself on? (Heaps, tries, binary search on the answer, the median problem.)
- What did the failure drills teach me about my own design?
- What will I change in my routine for Week 5, when cloud, containers and Kubernetes begin?

6. Week 5 Preview and Preparation

Week 5: cloud, Docker, CI/CD and Kubernetes

- **Day 1:** AWS core (IAM, EC2, VPC basics, security groups). DSA: Number of Islands, Clone Graph, BFS and DFS templates.
- **Day 2:** S3, RDS, ElastiCache, SQS and SNS. DSA: Course Schedule and topological sort.
- **Day 3:** Lambda, ECS and EKS overview, load balancers, CloudWatch. DSA: Rotting Oranges, Pacific Atlantic Water Flow.
- **Day 4:** Docker multi-stage builds and networking, CI/CD with Jenkins or GitHub Actions. DSA: union find.
- **Day 5:** Kubernetes (Pods, Deployments, Services, ConfigMaps and Secrets, probes, HPA). DSA: Dijkstra.
- **Day 6:** deploy your project to AWS through the pipeline.
- **Day 7:** system design of a notification service, plus a mock interview.

Do these before Week 5 starts

- **Create an AWS account safely.** Turn on **multi-factor authentication for the root user**, create an **IAM admin user** for daily work and stop using the root account, and set a **billing budget with an alert** (for example 5 or 10 dollars) before creating anything.
- Note which services are free tier, and remember that **running resources cost money**, so delete or stop everything at the end of each session.
- Install the **AWS CLI**, **kubectl**, and a local cluster tool (**kind** or **minikube**), and confirm that Docker works.
- If you can, ask a friend or colleague with AWS experience for a quick review of your setup.

7. End-of-Day Checklist

- [ ] I can recite the seven HLD steps and use them on a new prompt.
- [ ] I designed the rate limiter aloud in 35 minutes and compared it with the guide.
- [ ] I can explain the token bucket choice, atomicity, sharding, local batching, fail-open and shadow-mode rollout.
- [ ] I completed the Week 4 mock and wrote my review notes.
- [ ] My project checkpoint is complete, or the gaps are written down with a plan.
- [ ] My AWS account is secured with MFA, an IAM user and a billing alert.

Next (Week 5, Day 1): AWS core (IAM, EC2, VPC, security groups), plus graph problems (Number of Islands, Clone Graph).